Widoki i tabele tymczasowe

Przemysław Starosta - 2026

Wstęp

Widoki w PostgreSQL są wirtualnymi tabelami, które są tworzone na podstawie zapytania SQL.

Nie przechowują one danych samodzielnie, ale prezentują dane z jednej lub wielu tabel w sposób zdefiniowany przez zapytanie

Widok standardowy

Rodzaje widoków:

W PostgreSQL wyróżniamy dwa główne typy widoków:

- **Widoki standardowe (Standard Views):**
  - Są to widoki, które są tworzone za pomocą komendy CREATE VIEW.
  - Ich zawartość jest obliczana za każdym razem, gdy widok jest odpytywany. Oznacza to, że zawsze pokazują najbardziej aktualne dane.
  - Mogą być używane do upraszczania złożonych zapytań, ukrywania złożoności bazowej struktury danych, a także do implementacji mechanizmów bezpieczeństwa poprzez ograniczanie dostępu do określonych kolumn lub wierszy.
  - Nie poprawiają wydajności zapytań, ponieważ zapytanie bazowe jest wykonywane za każdym razem.

Widok standardowy

	last_name	salary	department_name	job_title
1	Martinez	13,000	Marketing	Marketing Manager
2	Davis	6,000	Marketing	Marketing Representative

```
create or replace view v_dept_20
as
select
    e.last_name,
    e.salary,
    d.department_name,
    j.job_title
from employees e join departments d on e.department_id = d.department_id
                 join jobs j on e.job_id = j.job_id
where e.department_id = 20


select * from v_dept_20;
```

Widok zmaterializowany

- **Widoki zmaterializowane (Materialized Views):**
  - Są to widoki, które przechowują wyniki zapytania fizycznie na dysku.
  - Są tworzone za pomocą komendy CREATE MATERIALIZED VIEW.
  - Ich zawartość nie jest aktualizowana automatycznie. Aby odświeżyć dane, należy użyć komendy REFRESH MATERIALIZED VIEW.
  - Główną zaletą widoków zmaterializowanych jest poprawa wydajności zapytań, zwłaszcza dla złożonych zapytań agregujących lub zapytań obejmujących duże zbiory danych, ponieważ dane są już wstępnie obliczone.
  - Wadą jest to, że dane mogą być nieaktualne, jeśli nie są regularnie odświeżane.

Widok zmaterializowany

	department_name	dept_salary
1	Accounting	20,308
2	Purchasing	24,900
3	Shipping	156,400
4	Human Resources	6,500
5	Marketing	19,000
6	Sales	377,140
7	Public Relations	10,000
8	Administration	4,400
9	IT	28,800
10	Executive	58,000
11	Finance	51,608

```
create materialized view mv_sum_dept
as
select
        d.department_name,
        sum(salary + (salary * coalesce(commission_pct,0))) dept_salary
from employees e join departments d on e.department_id = d.department_id
group by d.department_name


select * from mv_sum_dept;
```

Różnice

```
explain select * from v_dept_20;
```

	QUERY PLAN
1	Nested Loop (cost=0.15..21.42 rows=2 width=177)
2	-> Nested Loop (cost=0.00..5.06 rows=2 width=97)
3	-> Seq Scan on departments d (cost=0.00..1.70 rows=1 width=90)
4	Filter: (department_id = '20'::numeric)
5	-> Seq Scan on employees e (cost=0.00..3.34 rows=2 width=24)
6	Filter: (department_id = '20'::numeric)
7	-> Index Scan using job_id_pk on jobs j (cost=0.15..8.17 rows=1 width=126)
8	Index Cond: ((job_id)::text = (e.job_id)::text)

```
explain select * from mv_sum_dept;
```

	QUERY PLAN
1	Seq Scan on mv_sum_dept (cost=0.00..15.90 rows=590 width=110)

Widok zmaterializowany

zwykły
vs
[concurrently]

Różnica między REFRESH MATERIALIZED VIEW a REFRESH MATERIALIZED VIEW CONCURRENTLY

Obie komendy służą do odświeżania danych w widoku zmaterializowanym, ale robią to w zupełnie inny sposób, co ma ogromne znaczenie dla dostępności aplikacji.

- **REFRESH MATERIALIZED VIEW (zwykły)**
  - **Działanie:** Ta komenda blokuje dostęp do widoku zmaterializowanego na czas jego odświeżania. Oznacza to, że żadne zapytania SELECT nie mogą być wykonywane na tym widoku, dopóki proces odświeżania się nie zakończy.
  - **Proces:**
    - PostgreSQL usuwa wszystkie stare dane z widoku.
    - Wykonuje zapytanie bazowe widoku.
    - Wstawia nowe dane do widoku.
    - Tworzy na nowo indeksy (jeśli istnieją).
  - **Wymagania:** Nie wymaga żadnych specjalnych indeksów.
  - **Zalety:** Prostszy w użyciu, nie wymaga dodatkowych indeksów.
  - **Wady:**
    - Blokuje widok: Jest to poważny problem dla aplikacji produkcyjnych, gdzie nawet krótka niedostępność widoku może być nieakceptowalna.
    - Długie odświeżanie dużych widoków może prowadzić do długich przestojów.
  - **Kiedy używać:**
    - W środowiskach deweloperskich/testowych.
    - Dla małych widoków, których odświeżanie trwa ułamki sekund.
    - Gdy akceptowalny jest krótki przestój w dostępie do danych.

Widok zmaterializowany

zwykły
vs
[concurrently]

- **REFRESH MATERIALIZED VIEW CONCURRENTLY**
  - **Działanie:** Ta komenda pozwala na odświeżanie widoku zmaterializowanego bez blokowania dostępu do niego dla zapytań SELECT. Użytkownicy mogą nadal odpytywać widok, widząc jego stare dane, podczas gdy w tle trwa proces odświeżania. Po zakończeniu odświeżania, widok atomowo przełącza się na nowe dane.
  - **Proces:**
    - PostgreSQL tworzy tymczasową kopię widoku zmaterializowanego (np. mv_temp_XXXX).
    - Wykonuje zapytanie bazowe widoku i wstawia nowe dane do tej tymczasowej kopii.
    - Porównuje stare i nowe dane, aby zidentyfikować zmiany.
    - Wykonuje operacje INSERT, UPDATE, DELETE na oryginalnym widoku, aby zsynchronizować go z nowymi danymi.
    - Atomowo zamienia stary widok na nowy (lub aktualizuje go w miejscu).
  - **Wymagania:**
    - Musi istnieć unikalny indeks na widoku zmaterializowanym. Ten indeks jest używany do identyfikacji wierszy podczas porównywania starych i nowych danych. Bez niego PostgreSQL nie wie, jak efektywnie porównać i zaktualizować dane.
    - Nie może być żadnych otwartych transakcji, które odpytują widok zmaterializowany w momencie rozpoczęcia REFRESH CONCURRENTLY.

Widok zmaterializowany

zwykły
vs
[concurrently]

- **REFRESH MATERIALIZED VIEW CONCURRENTLY**
  - **Zalety:**
    - Brak blokady odczytu: Aplikacje mogą nadal działać i odczytywać dane z widoku podczas odświeżania.
    - Idealne dla systemów produkcyjnych o wysokiej dostępności.
  - **Wady:**
    - Wymaga unikalnego indeksu.
    - Może być wolniejsze niż zwykły REFRESH dla bardzo dużych widoków, ponieważ wymaga dodatkowych operacji (tworzenie tymczasowej kopii, porównywanie danych, aktualizacja wiersz po wierszu).
    - Zużywa więcej zasobów (miejsca na dysku dla tymczasowej kopii, CPU dla porównywania).
  - **Kiedy używać:**
    - W systemach produkcyjnych, gdzie dostępność jest krytyczna.
    - Dla dużych widoków, których odświeżanie trwa długo.
    - Gdy nie można sobie pozwolić na blokowanie odczytu.

```
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales_summary;
```

Widok zmaterializowany

zwykły
vs
[concurrently]

Podsumowanie różnic

Cecha	REFRESH MATERIALIZED VIEW	REFRESH MATERIALIZED VIEW CONCURRENTLY
Blokada odczytu	**TAK** (widok jest zablokowany)	**NIE** (widok jest dostępny do odczytu)
Wymagany indeks	Nie	**TAK** (unikalny indeks)
Szybkość odświeżania	Zazwyczaj szybszy	Zazwyczaj wolniejszy (więcej operacji)
Zużycie zasobów	Mniejsze	Większe (tymczasowa kopia, porównywanie)
Zastosowanie	Dev/test, małe widoki, akceptowalny przestój	Produkcja, duże widoki, wysoka dostępność

Tabele tymczasowe

Czym są tabele tymczasowe?

- Tabele tymczasowe (TEMP lub TEMPORARY) w PostgreSQL to tabele, które istnieją tylko w ramach sesji albo transakcji, w której zostały utworzone.
- Po jej zakończeniu tabela jest automatycznie usuwana, razem z wszystkimi danymi.
- Dzięki temu można bezpiecznie operować na tymczasowych danych, które nie zaśmiecą schematu bazy.
- Tworzy się je bardzo podobnie jak zwykłe tabele, z tą różnicą, że używa się słowa kluczowego TEMP lub TEMPORARY:

```
CREATE TEMP TABLE temp_orders (
order_id SERIAL,
customer_id INT,
total_amount NUMERIC(10,2));
```

```
CREATE TEMPORARY TABLE temp_orders (
order_id SERIAL,
customer_id INT,
total_amount NUMERIC(10,2));
```

Tabele tymczasowe

W PostgreSQL są dwa tryby życia takiej tabeli:

- **Domyślnie: ważna dla całej sesji**
  - Jeśli napiszesz *CREATE TEMP TABLE ... ON COMMIT PRESERVE ROWS* → dane zostają w tabeli przez całą sesję.
- **Tymczasowe na czas transakcji**
  - Jeśli napiszesz *CREATE TEMP TABLE ... ON COMMIT DELETE ROWS*,→ po COMMIT dane zostają usunięte, ale schema i kolumny tabeli nadal istnieją.
  - Jeśli użyjesz *ON COMMIT DROP*,→ cała tabela zostaje usunięta automatycznie po zakończeniu transakcji.

```
CREATE TEMP TABLE temp_session_data (
user_id INT,
last_action TIMESTAMP) ON COMMIT DELETE ROWS;
```

Tabele tymczasowe

Kluczowe cechy:

- **Widoczność:** tabela tymczasowa jest widoczna tylko dla bieżącej sesji – inni użytkownicy jej nie zobaczą.
- **Nazwy:** możesz utworzyć tabelę tymczasową o tej samej nazwie co tabela stała – w tej sesji zawsze będzie brana pod uwagę tabela tymczasowa.
- **Indeksy i constraints:** możesz zakładać indeksy i ograniczenia tak jak w zwykłych tabelach.
- **Wydajność:** dane są przechowywane w pamięci lub na dysku (jak normalne tabele), ale optymalizator wie, że są lokalne dla sesji, co czasem daje mu prostsze plany wykonania.
- **Autovacuum:** nie działa na tabele tymczasowe – ich czyszczenie obsługuje mechanizm sesji

Tabele tymczasowe

Kiedy używać tabel tymczasowych?

✅ Scenariusze użyteczne:

- przetwarzanie danych pośrednich w złożonych raportach / ETL,
- trzymanie wyników kroków algorytmu,
- manipulacja i testowanie danych w sesji roboczej,
- przyspieszenie złożonych zapytań, kiedy wielokrotnie potrzebujesz tych samych (przetworzonych) danych w jednej sesji.

❌ Unikać, jeśli:

- potrzebujesz współdzielić dane między różnymi sesjami użytkowników,
- tabela ma być persystentna(trwała).

Porównanie widoków materializowanych i tabel tymczasowych (1/5)

- **Podobieństwa**
  - Oba przechowują dane fizycznie, a nie tylko definicję zapytania jak zwykły widok.
  - Mogą przyspieszać złożone raporty, bo możesz wykonać kosztowne zapytanie raz i korzystać z wyników później.
  - Mogą mieć indeksy, klucze itd.
  - Dane nie są automatycznie odświeżane – wymagają albo ręcznego załadowania (INSERT/SELECT w tabelach tymczasowych), albo REFRESH w przypadku widoków zmaterializowanych.

Porównanie widoków materializowanych i tabel tymczasowych (2/5)

Cecha	Tabela tymczasowa	Widok zmaterializowany
Czas życia	Do końca **sesji** albo transakcji (wg opcji ON COMMIT)	Trwała struktura w bazie – istnieje aż do DROP MATERIALIZED VIEW
Widoczność	Tylko w sesji, która utworzyła	Wspólny dla wszystkich użytkowników (wg uprawnień)
Zawartość	Wypełniasz ręcznie (INSERT, CREATE … AS SELECT)	Tworzona automatycznie przy CREATE MATERIALIZED VIEW, potem odświeżana komendą REFRESH
Aktualność danych	Zależy od zapytań, którymi ładujesz dane	Dane mogą być **nieaktualne**, dopóki nie zrobisz REFRESH
Odświeżanie	Poprzez nowe INSERT/UPDATE/DELETE	REFRESH MATERIALIZED VIEW [CONCURRENTLY]

Porównanie widoków materializowanych i tabel tymczasowych (3/5)

Cecha	Tabela tymczasowa	Widok zmaterializowany
Kolizja nazw	Może istnieć obok zwykłej tabeli o tej samej nazwie w tej samej bazie (ale w sesji widoczna jest temp)	Nazwa jest globalna – jak zwykła tabela, tylko jedna w bazie
Zastosowania	Dane pomocnicze na czas zapytania, ETL, operacje testowe, cache "na chwilę"	Cache wyników raportów, "snapshoty" dużych danych, przyspieszenie ciężkich analiz
Indeksy	Można tworzyć (jak normale tabele)	Można tworzyć (wymagane np. do REFRESH CONCURRENTLY)
Autovacuum	Nie dotyczy – temp tabele czyszczone automatycznie	Działa jak na normalnych tabelach
Wydajność	Bardzo szybkie w sesji (prywatny kontekst), brak narzutu transakcji dla innych	Koszt REFRESH może być duży przy dużych danych, ale potem odpytywanie bardzo szybkie

Porównanie widoków materializowanych i tabel tymczasowych (4/5)

- Kiedy używać?

✅ Tabela tymczasowa:

- masz dane „na chwilę” (np. w raportach wieloetapowych albo w ETL-u),
- dane są unikalne dla sesji użytkownika (np. koszyk zakupów przed zatwierdzeniem),
- chcesz szybko przetestować wyniki i nie zaśmiecać bazy.

✅ Widok zmaterializowany:

- chcesz trwale przechowywać snapshot wyników, np. sprzedaży dziennych,
- te same dane mają być widoczne dla wielu użytkowników,
- koszt REFRESH jest mniejszy niż wielokrotne wykonywanie ciężkiego zapytania.

Porównanie widoków materializowanych i tabel tymczasowych (5/5)

- **Tabela tymczasowa** → jak notes, w którym zapisujesz coś tylko na czas spotkania. Po wyjściu wyrzucasz kartkę.
- **Widok zmaterializowany** → jak wydrukowany raport – może być nieaktualny, ale każdy w firmie dostaje tę samą wersję, dopóki nie wydrukujesz nowej.